



Introducing Code Quality in the CS1 Classroom

Cruz Izu*
The University of Adelaide
Adelaide, Australia
cruz.izu@adelaide.edu.au

Claudio Mirolo*
University of Udine
Udine, Italy
claudio.mirolo@uniud.it

Jürgen Börstler
Blekinge Institute of Technology
Karlskrona, Sweden
jurgen.borstler@bth.se

Harold Connamacher
Case Western Reserve University
Cleveland, OH, USA
harold.connamacher@case.edu

Ryan Crosby
Durham University
Durham, England
ryan.crosby@durham.ac.uk

Richard Glassey
KTH Royal Institute of Technology
Stockholm, Sweden
glassey@kth.se

Georgiana Haldeman
Colgate University
Hamilton, NY, USA
ghaldeman@colgate.edu

Olli Kiljunen
Aalto University
Helsinki, Finland
olli.kiljunen@aalto.fi

Amruth N. Kumar
Ramapo College of New Jersey
Mahwah, NJ, USA
amruth@ramapo.edu

David Liu
University of Toronto
Toronto, Canada
david@cs.toronto.edu

Andrew Luxton-Reilly
University of Auckland
Auckland, New Zealand
andrew@cs.auckland.ac.nz

Stephanos Matsumoto
Olin College of Engineering
Needham, MA, USA
smatsumoto@olin.edu

Eduardo Carneiro de Oliveira
Utrecht University
Utrecht, Netherlands
e.carneirodeoliveira@uu.nl

Seán Russell
University College
Dublin, Ireland
sean.russell@ucd.ie

Anshul Shah
University of California, San Diego
San Diego, CA, USA
ayshah@ucsd.edu

ABSTRACT

Characterising code quality is a challenge that was addressed by Börstler et al.'s working group in 2017. As emerged from their study, educators, developers and students have different perceptions of the manifold aspects involved, and a major conclusion of that WG was that “code quality should be discussed more thoroughly in educational programs” [2, p. 70]. However, the lack of materials and the time constraints have slowed down progress in that regard.

The goal of this working group is to propose manageable ways to address code quality in the CS1 classroom, with a particular focus on activities that help students become aware of and improve the quality of their code. To achieve this goal, we will (a) extract from the literature a comprehensive set of quality issues which will then be classified according to the appropriate strategies to fix them; and (b) circulate a survey to explore the instructors' views on code quality issues and the way they deal with (or ignore) them. Based on this work we aim to produce: (1) a taxonomy of code quality issues with associated examples, as well as (2) a sample set of teaching materials to introduce those issues to CS1 students.

*co-leader

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
ITiCSE 2024, July 8–10, 2024, Milan, Italy
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0603-5/24/07.
<https://doi.org/10.1145/3649405.3659535>

CCS CONCEPTS

• **Social and professional topics** → CS1.

KEYWORDS

CS1, code quality, readability, style, refactoring

ACM Reference Format:

Cruz Izu, Claudio Mirolo, Jürgen Börstler, Harold Connamacher, Ryan Crosby, Richard Glassey, Georgiana Haldeman, Olli Kiljunen, Amruth N. Kumar, David Liu, Andrew Luxton-Reilly, Stephanos Matsumoto, Eduardo Carneiro de Oliveira, Seán Russell, and Anshul Shah. 2024. Introducing Code Quality in the CS1 Classroom. In *Proceedings of the 2024 Innovation and Technology in Computer Science Education V. 2 (ITiCSE 2024)*, July 8–10, 2024, Milan, Italy. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3649405.3659535>

1 INTRODUCTION AND MOTIVATION

Education research regarding novice programmers has documented the difficulties that students experience when writing functional code and the misconceptions that cause incorrect code. However, as pointed out by Gaber and Kirsh, “[a]lthough students are aware of many of their bugs and missing features, they are not sufficiently aware of the quality of their code” [4, p. 79].

Börstler et al. [2] investigated the perceptions of code quality by interviewing students, instructors and professional developers: participants would bring their own good or bad code examples and answer a list of questions regarding features and factors that determine code quality. They observed that students, educators and developers placed different emphasis on the wide variety of

considered aspects: coding style/formatting, naming conventions, readability, comprehensibility, documentation, code structure, duplications/redundancies, modularity, dependencies/coupling, performance, robustness, maintainability, testability, etc.

Indeed, while correctness and performance can be established objectively, other criteria, such as readability, are subjective and allow for multiple — even opposite — standpoints [9]. In particular, grading the quality of small programs is challenging and partly subjective [6]. Thus, each individual may perceive code quality differently; for example, in contrast to experienced programmers, novices often find verbose code to be more readable [9]. Additionally, different groups will weight quality criteria differently [2] so that even when their perceptions of isolated aspects match, their overall quality assessment would differ.

Keuning et al. have recently conducted a systematic literature review on code quality in education at all levels. Based on the large dataset they collected, it emerges that “much of the program quality research appeared in the last decade” [7, p. 10]. The authors conclude that “a possible direction for future work is to conduct a more in-depth literature study of” some more specific topics, and “encourage researchers to perform studies on the topics that have received little attention so far, such as integrating code quality into the computing curricula, developing and evaluating course materials” [7, p. 10]. In fact, Kirk et al. [8] reported that only 41 of 141 CS1 courses syllabi in 2020 mentioned the term “code quality”. In a similar vein, Börstler et al. observed that “students [...] get less information regarding code quality from their education than from other students or the Internet. [...] Giving students more exposure to issues regarding code quality as part of their education [...] could solve some of these issues” [2, p. 82].

In short, we should aspire to teach undergraduate students to write code that is both correct and exhibits *good quality*. Recent efforts to improve the quality of CS1 code include identifying a range of code quality issues [3], developing tools such as *Hyperstyle* [1] that provide a code quality score, and producing simple resources to introduce refactoring of conditional statements [5]. Nonetheless, more teaching materials are needed that address code quality at CS1 level, both to raise awareness and to help students improve the quality of their code.

2 WG METHODOLOGY AND OUTCOMES

The main objective of this working group is to provide worked examples and associated learning resources that will help instructors discuss code quality issues with their students. Additionally, a taxonomy will standardise language and fundamental quality topic ideas, providing a conceptual framework that instructors can use to discuss code quality.

2.1 Working plan

To develop worked examples, we need to collect a comprehensive list of quality issues and then classify them in relation to the strategies needed to eliminate them. We will also look for learning resources from the literature and CS1 instructors. Therefore, the Working Group (WG) will direct their collaborative effort towards three main tasks as described next.

Literature review: the WG will review the literature to identify the reported quality issues at the introductory level. The recent systematic review by Keuning et al. on code quality in CS education will play a significant role in this step; it provides 195 source papers on the topic, to be complemented with some recent works.

Instructor survey: the WG will design and deliver a survey addressed to CS instructors to capture their views and approaches regarding code quality instruction and if/how they assess quality issues in students’ submissions.

Quality issues taxonomy: based on the outcome of the previous two sub-tasks, the emerging quality issues will be classified in terms of involved programming constructs, significance of the issue, and potential strategies to remedy them. Consensus will be sought from the group members, representing CS, SE and IT fields on terminology and classification and whether the issue needs to be addressed in CS1 or later.

2.2 Expected Outcomes

The Working Group report will be focused on providing two deliverables to support CS1 instructors in discussing code quality:

- A taxonomy of code quality issues/defects, with a list of associated examples.
- A “catalogue” of sample teaching materials that address quality issues by presenting strategies to improve code design, code structure, and code style.

REFERENCES

- [1] Anastasiia Birillo, Ilya Vlasov, Artyom Burylov, Vitalii Selishchev, Artyom Goncharov, Elena Tikhomirova, Nikolay Vyahhi, and Timofey Bryksin. 2022. *Hyperstyle: A Tool for Assessing the Code Quality of Solutions to Programming Assignments*. In *Proc. of the 53rd ACM Tech. Symposium on Computer Science Education (SIGCSE 2022)*. ACM, New York, NY, USA, 307–313.
- [2] Jürgen Börstler, Harald Störrle, Daniel Toll, Jelle van Assema, Rodrigo Duran, Sara Hooshangi, Johan Jeuring, Hieke Keuning, Carsten Kleiner, and Bonnie MacKellar. 2018. “I Know It When I See It” Perceptions of Code Quality: ITiCSE ’17 Working Group Report. In *Proc. of the 2017 ITiCSE Conference on Working Group Reports (ITiCSE-WGR ’17)*. ACM, New York, NY, USA, 70–85.
- [3] Tomáš Effenberger and Radek Pelánek. 2022. Code Quality Defects across Introductory Programming Topics. In *Proc. of the 53rd ACM Tech. Symposium on Computer Science Education (SIGCSE 2022)*. ACM, New York, NY, USA, 941–947.
- [4] Iris Gaber and Amir Kirsh. 2018. The Effect of Reporting Known Issues on Students’ Work. In *Proc. of the 49th ACM Tech. Symposium on Computer Science Education (SIGCSE ’18)*. ACM, New York, NY, USA, 74–79.
- [5] Cruz Izu, Paul Denny, and Sayoni Roy. 2022. A Resource to Support Novices Refactoring Conditional Statements. In *Proc. of the 27th ACM Conf. on Innovation and Technology in Computer Science Education (ITiCSE ’22)*. ACM, New York, NY, USA, 344–350.
- [6] Remin Kasahara, Kazunori Sakamoto, Hironori Washizaki, and Yoshiaki Fukazawa. 2019. Applying Gamification to Motivate Students to Write High-Quality Code in Programming Assignments. In *Proc. of the 2019 ACM Conf. on Innovation and Technology in Computer Science Education (ITiCSE ’19)*. ACM, New York, NY, USA, 92–98.
- [7] Hieke Keuning, Johan Jeuring, and Bastiaan Heeren. 2023. A Systematic Mapping Study of Code Quality in Education. In *Proc. of the 2023 Conf. on Innovation and Technology in Computer Science Education (ITiCSE 2023)*. ACM, New York, NY, USA, 5–11.
- [8] Diana Kirk, Tyne Crow, Andrew Luxton-Reilly, and Ewan Tempero. 2022. Teaching Code Quality in High School Programming Courses - Understanding Teachers’ Needs. In *Australasian Computing Education Conference (ACE ’22)*. ACM, New York, NY, USA, 36–45.
- [9] Eliane S Wiese, Anna N Rafferty, and Armando Fox. 2019. Linking code readability, structure, and comprehension among novices: it’s complicated. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*. IEEE, 84–94.